

Advanced Programming Lab 3

Functional Programming in Python

10 Questions, each worth 10 points

Total Points: 100

Contents

1	Introduction	2
2	Files	2
3	Provided Starter Code	2
4	General Instructions	3
5	Implementation Rules	3
6	Questions	3
7	Grading Criteria	9
8	Submission Checklist	9

Learning Outcomes. After completing this lab, students should be able to:

1. Apply Church encodings and higher-order functions in Python.
2. Use recursion and fixed-point combinators to express repeated computation.
3. Reason about simple typed lambda-calculus expressions through type checking.
4. Use divide-and-conquer techniques on linked-list data structures.
5. Implement folds over lists and trees.
6. Relate currying and function structure to Curry–Howard style ideas.

1 Introduction

This lab brings together the main functional-programming ideas covered in class into one consolidated worksheet. Students will work with Church booleans, higher-order functions, recursive list processing, fixed-point combinators, typed-expression checking, linked-list algorithms, folds, and currying. The questions are arranged from shorter conceptual tasks to more structured recursive problems so that each part builds on ideas introduced earlier in the lab.

2 Files

1. `lab_10_questions.py`: This is the main file you must complete. It contains all 10 questions, the provided helper definitions, and the `TODO` locations where you will write your code.

3 Provided Starter Code

You are provided with a starter Python file containing function declarations, class definitions, helper utilities, and `NotImplementedError` placeholders. Your task is to replace the `TODO` implementations with working code while preserving the existing public structure of the file.

Important. Do not modify function names, method signatures, class names, or the provided data structures unless the worksheet explicitly asks you to do so. Your solutions should fit into the given scaffold.

4 General Instructions

1. Open `lab_10_questions.py` and read through the full file before you begin coding so you understand the provided helper functions, classes, and data structures.
2. Implement the questions in order from Q1 to Q10. The questions are arranged to move from shorter functional-programming tasks to larger recursive and data-structure problems.
3. Write your solution only in the provided `TODO` areas. Keep all function names, parameter lists, and return expectations unchanged so your work remains compatible with the lab structure.
4. Do not remove, rename, or redesign the provided classes, helper functions, or data structures unless the lab explicitly asks you to do so.
5. Use the examples under each question to sanity-check your logic as you work. These examples are intended to help you confirm expected behavior, but you should also test additional cases of your own.
6. If a question involves recursion or a functional-programming pattern, try to solve it using the intended idea rather than replacing it with an unrelated shortcut.
7. Each question is worth **10 points**, for a total of **100 points**.

5 Implementation Rules

Area	Key Rules
Function questions	Keep the provided function signatures unchanged and return values in the expected format.
Recursive questions	Use recursion where the question explicitly requires it, especially for <code>map_recursive</code> , <code>fix</code> , and linked-list/tree processing tasks.
Typed expressions	Follow the provided expression and type classes exactly when implementing the type checker.
Linked lists	Work with the linked-list nodes directly rather than converting the entire structure into a Python list to solve the main algorithm.
Folds and currying	Use the functional pattern requested in the question rather than replacing it with an unrelated shortcut.

6 Questions

Q1. Church Boolean NOT

Implement `church_not`. Write a function that takes a Church-encoded boolean and returns its logical negation. Your implementation should preserve the Church encoding style, meaning it should return another function that swaps the true and false branches.

In other words, if the input chooses the first argument, the output should choose the second, and vice versa.

Requirements

1. Return a Church-encoded boolean.
2. Do not convert to Python `True` or `False` internally.
3. The function must correctly reverse both `church_true` and `church_false`.

Example 1

Input: `from_church_bool(church_not(church_true))`
Output: `False`

Example 2

Input: `church_not(church_false)("left")("right")`
Output: `"left"`

Q2. Variadic Composition

Implement `compose(f, g)` so the result computes `f(g(*args, **kwargs))`. Create a higher-order function that returns a new function composing `f` after `g`. The key requirement is that the returned function must accept any positional and keyword arguments, pass them to `g`, and then send the single result of `g` into `f`. This question checks your understanding of flexible argument passing and function composition in Python.

Requirements

1. The returned function must accept both `*args` and `**kwargs`.
2. Call `g` first, then pass its single result into `f`.
3. Do not change the function signature of `compose`.

Example 1

Input: `compose(lambda x: x + 1, lambda x: x * 2)(10)`
Output: `21`

Example 2

Input: `compose(lambda s: s.upper(), lambda a,b,sep="-": f"{a}{sep}{b}")( "a", "b", sep=":")`
Output: `"A:B"`

Q3. Recursive Map

Implement `map_recursive(fn, xs)` recursively. Implement your own version of `map` without using loops or Python's built-in `map`. The function should apply `fn` to every element in the list `xs` and return a new list with the transformed values. Be sure to handle the base case correctly so that the recursion stops on an empty list.

Requirements

1. Use recursion rather than `for` or `while`.
2. Return a new list and leave the original input list unchanged.
3. Correctly handle the empty-list base case.

Example 1

```
Input: map_recursive(lambda x: x*x, [1,2,3])
Output: [1, 4, 9]
```

Example 2

```
Input: map_recursive(lambda x: x+10, [])
Output: []
```

Q4. Fixed-Point Combinator

Implement `fix` (Z-style) for eager Python. Write a fixed-point combinator that enables anonymous recursive functions in eager evaluation. Since standard Y combinators do not work directly in Python, this version should follow the Z-combinator style so recursive calls are delayed correctly. The result should allow a function to refer to itself through a `rec` parameter without naming the final function explicitly.

Requirements

1. Use an eager-language-safe fixed-point style suitable for Python.
2. The returned function must support recursive calls through the provided `rec` parameter.
3. Your implementation should work for functions with one or more arguments.

Example 1

```
Input: fix(lambda rec: lambda n: 1 if n==0 else n*rec(n-1))(5)
Output: 120
```

Example 2

```
Input: fix(lambda rec: lambda a,b: a if b==0 else rec(b,a%b))(48,18)
Output: 6
```

Q5. Typed Lambda Calculus: Type Checker

Implement `type_of` for literals, lambdas, application, `If`, `Add`, `Sub`, `Mod`, and `IsZero`. Build a type checker for the provided expression language. Your function should examine the structure of an expression and return the correct type when the expression is valid. You will need to handle integer and boolean literals, function abstractions and applications, conditional expressions, arithmetic operators, and zero-testing. Make sure your checker also enforces type consistency, such as requiring arithmetic operands to be integers and both branches of an `If` expression to have the same type.

Requirements

1. Support all listed expression forms.
2. Enforce correct operand and branch types.
3. Return the appropriate type object for valid expressions.
4. Raise or report a type error for invalid expressions according to the starter code design.

Example 1

```
Input: type_of(Add(IntLit(3), IntLit(2)))
Output: TInt()
```

Example 2

```
Input: type_of(If(BoolLit(True), IntLit(1), IntLit(2)))
Output: TInt()
```

Q6. Merge Two Sorted Linked Lists

Implement `merge_sorted_lists(a, b)`. Given two already sorted linked lists, merge them into a single sorted linked list. Reuse the existing nodes rather than converting to regular Python lists. Your solution should correctly handle edge cases such as one or both input lists being empty, and it should preserve all values in nondecreasing order.

Requirements

1. Assume both input lists are already sorted in nondecreasing order.
2. Reuse linked-list nodes instead of converting to Python lists.
3. Correctly handle `None` inputs.

4. Preserve duplicates in the merged result.

Example 1

```
Input:      ll_to_list(merge_sorted_lists(ll_from_iterable([1,3,5]),  
ll_from_iterable([2,4,6])))  
Output: [1, 2, 3, 4, 5, 6]
```

Example 2

```
Input: ll_to_list(merge_sorted_lists(None, ll_from_iterable([7,8])))  
Output: [7, 8]
```

Q7. Merge Sort on Linked List

Implement `merge_sort_linked(head)`. Implement merge sort for a singly linked list. This requires splitting the list into two halves, recursively sorting each half, and then merging the sorted halves back together. Because linked lists do not support direct indexing, think carefully about how to find the midpoint efficiently and how to reconnect nodes after splitting.

Requirements

1. Use the merge-sort strategy: split, recursively sort, then merge.
2. Work directly on the linked-list structure.
3. Correctly handle empty and single-node lists.
4. Return the head of the sorted linked list.

Example 1

```
Input: ll_to_list(merge_sort_linked(ll_from_iterable([4,2,9,1,2])))  
Output: [1, 2, 2, 4, 9]
```

Example 2

```
Input: ll_to_list(merge_sort_linked(None))  
Output: []
```

Q8. Left Fold for Iterables

Implement `foldl(fn, init, xs)`. Write a left fold that processes the iterable `xs` from left to right. Starting with the accumulator value `init`, repeatedly combine the current accumulator and the next element using `fn`. This question focuses on reducing a sequence into a single result while preserving left-to-right evaluation order.

Requirements

1. Process items from left to right.
2. Start with `init` as the initial accumulator.
3. Return the final accumulated value.
4. Work for general iterables, not only lists.

Example 1

```
Input: foldl(lambda acc,x: acc+x, 0, [1,2,3,4])  
Output: 10
```

Example 2

```
Input: foldl(lambda acc,x: acc+x, "", ["a","b","c"])  
Output: "abc"
```

Q9. Fold Over Binary Tree

Implement `fold_tree(leaf, node_fn, t)`. Generalize the folding idea from lists to binary trees. If the tree is empty, return the `leaf` value. Otherwise, recursively fold the left and right subtrees and combine those results with the current node's value using `node_fn`. A correct implementation can be used to express different tree algorithms such as traversal, counting, or aggregation.

Requirements

1. Return `leaf` when the tree is empty.
2. Recursively fold both left and right subtrees.
3. Combine the current value and subtree results with `node_fn`.
4. Keep the implementation generic enough to support multiple tree computations.

Example 1

```
Input: fold_tree([], lambda v,l,r: 1+[v]+r, Tree(2, Tree(1),  
Tree(3)))  
Output: [1, 2, 3]
```

Example 2

```
Input: fold_tree(0, lambda _v,l,r: 1+l+r, Tree(5, Tree(2), Tree(8)))  
Output: 3
```


Q10. Currying (Curry–Howard Style)

Implement `q5_curry` with type shape $((A, B) \rightarrow C) \rightarrow A \rightarrow B \rightarrow C$. Transform a function that takes a pair into a curried function that takes its arguments one at a time. The returned function should first accept a value of type `A`, then return another function that accepts a value of type `B`, and finally apply the original function to the pair `(A, B)`. This question connects functional programming practice with the Curry–Howard correspondence discussed in class.

Requirements

1. Return nested unary functions.
2. The original function should receive a pair/tuple containing both inputs.
3. Preserve the order of arguments: first `A`, then `B`.
4. Match the target type shape $((A, B) \rightarrow C) \rightarrow A \rightarrow B \rightarrow C$.

Example 1

Input: `q5_curry(lambda p: f"{p[0]}:{p[1]}")("A")("B")`
Output: `"A:B"`

Example 2

Input: `q5_curry(lambda p: p[0] + p[1])(10)(5)`
Output: `15`

7 Grading Criteria

Criteria	Points
Q1 through Q10, 10 points each	100

8 Submission Checklist

1. All 10 TODOs are implemented in `lab_10_questions.py`.
2. Outputs match the examples in this sheet.
3. Code is readable and consistent.